CAT AI

Complete Architecture Blueprint

Production-Grade AI Platform

Version 1.0 · 2026

Conversational AI · Workflow Automation · RAG Knowledge Base · SaaS Platform

© 2026 CAT AI · Confidential

Table of Contents

Table of Contents 2
1. Product Overview 5
What CAT AI Is 5
Core Value Proposition 5
Target Users 5
How CAT AI Differs from ChatGPT / Claude / Grok 5
MVP Scope 5
Future Roadmap 6
2. Core Product Modules 7
3. High-Level System Architecture 8
Architecture Overview 8
ASCII Architecture Diagram 8
Synchronous vs Asynchronous Flows 8
Service Boundaries 9
4. Frontend Architecture 10
Next.js App Structure 10
Key UI Patterns 10
State Management 10
5. Backend / API Architecture 11
FastAPI Service Layout 11
Streaming Strategy 11
Auth & RBAC 11
Multi-Tenant Support 11
Rate Limiting 11
6. AI Orchestration Architecture 12
Prompt Assembly Pipeline 12
Model Routing Logic 12
Context Window Management 12
Response Streaming Design 12
Guardrails & Moderation 12
7. Workflow Automation Engine 13
Workflow Model 13
Step Types 13
Execution Engine Design 13
Human-in-the-Loop Approvals 14

8. Memory Architecture 15
Memory Extraction Pipeline 15
Memory Retrieval 15
Privacy Controls 15
9. RAG / Knowledge Base Architecture 16
File Ingestion Pipeline 16
Chunking Strategy 16
Retrieval Strategy 16
pgvector vs Qdrant Tradeoff 16
10. Database Architecture 18
PostgreSQL Data Model 18
When to Use Redis / S3 19
11. Model Provider Abstraction Layer 20
Adapter Interface 20
Provider Normalization 20
Circuit Breaker & Failover 20
Cost Tracking 20
12. Tool / Agent Framework 21
Tool Registry Design 21
Built-in Tools 21
Tool Execution Lifecycle 21
Prompt Injection Defense 22
13. Security Architecture 23
14. DevOps / Deployment Architecture 24
AWS Infrastructure 24
CI/CD Pipeline 24
Backup & Disaster Recovery 24
15. Observability & Analytics 25
16. Billing & Monetization Architecture 26
Plan Structure 26
Stripe Integration 26
17. Scalability Strategy 27
Key Scaling Decisions 27
18. Recommended Tech Stack 28
MVP Stack (Speed to Ship) 28
Production Stack (Full Platform) 28

19. Monorepo / Project Structure 29
20. Sample API Surface 30
21. End-to-End Request Flows 31
A. Normal Chat Request 31
B. Chat Request with Tool Call 31
C. Chat Request with RAG Retrieval 31
D. Workflow Triggered by Webhook 31
E. Scheduled Workflow Run 32
F. Human Approval Workflow Step 32
G. Memory Extraction After Response 32
22. Risks & Tradeoffs 33
23. MVP Phased Blueprint 34
Phase 1: Core Chat + Auth + Streaming (Weeks 1–3) 34
Phase 2: History + Model Routing + Workspaces (Weeks 4–5) 34
Phase 3: File Upload + RAG (Weeks 6–8) 34
Phase 4: Tool Calling + Integrations (Weeks 9–11) 34
Phase 5: Workflow Automation Engine (Weeks 12–15) 35
Phase 6: Billing + Analytics + Hardening (Weeks 16–18) 35
24. Final Recommendation 36
Architecture Summary 36
Best MVP Architecture 36
Best Production Architecture 36
Exact Frameworks & Libraries 36
First 30-Day Architecture-to-Build Roadmap 37

1. Product Overview

What CAT AI Is

CAT AI is a production-grade AI assistant and automation platform that unifies conversational AI, workflow automation, and knowledge retrieval into a single multi-tenant SaaS product. It is built to compete with and complement tools like ChatGPT, Claude.ai, and Grok while adding first-class workflow automation capabilities inspired by Zapier, n8n, and LangGraph.

Core Value Proposition

Pillar	Value Delivered
Unified AI Workspace	One platform for chat, automation, and knowledge — no context switching
Multi-Model Flexibility	Route queries to OpenAI, Anthropic, Groq, Gemini based on cost/quality/speed
Workflow Automation	Build trigger-based, multi-step AI agent workflows with conditional branching
Team Knowledge	Upload docs, build vector knowledge bases, get cited RAG responses
Enterprise-Ready	RBAC, audit logs, SSO path, usage metering, billing, GDPR compliance

Target Users

- Individual developers and power users who want a customizable AI assistant
- Small-to-medium teams needing shared workspaces, knowledge bases, and AI automations
- Product teams building internal AI tooling without full engineering investment
- Enterprises requiring compliance, RBAC, SSO, and audit logging

How CAT AI Differs from ChatGPT / Claude / Grok

Feature	ChatGPT / Claude / Grok	CAT AI
Multi-model routing	Single provider only	OpenAI + Anthropic + Groq + Gemini
Workflow automation	Limited or none	Full durable workflow engine
Team knowledge bases	Basic file upload	Full RAG pipeline with citations
RBAC / workspaces	Limited	First-class multi-tenant SaaS
Open integrations	Plugin-based (limited)	Webhook + OAuth integrations
Self-hosted model path	Not supported	Optional vLLM integration
Custom billing/plans	Fixed plans	Custom usage metering + Stripe

MVP Scope

- Multi-turn chat with streaming, chat history, workspace prompts
- Authentication, workspaces, basic RBAC
- Support for OpenAI and Anthropic (primary MVP models)
- File upload and basic RAG retrieval
- Usage tracking and basic billing integration

Future Roadmap

1. Phase 2: Groq/Gemini support, model routing, conversation summarization, memory
2. Phase 3: Full RAG pipeline, hybrid retrieval, reranking, citations
3. Phase 4: Tool calling, built-in tools, integration service
4. Phase 5: Workflow automation engine, durable execution, human approvals
5. Phase 6: Enterprise SSO, advanced analytics, LLM evaluation, compliance hardening

2. Core Product Modules

Module	Responsibility
Auth & User Management	JWT-based auth, OAuth (Google/GitHub), workspace membership, roles
Chat Interface	Streaming chat UI, message history, conversation branching, model selector
Conversation Service	CRUD for conversations/messages, summarization, context assembly
AI Orchestration Engine	Prompt assembly, token budgeting, model selection, response streaming
Model Provider Abstraction	Unified adapter for OpenAI/Anthropic/Groq/Gemini/vLLM
Tool Execution Engine	Tool registry, permission checks, sandbox execution, trace logging
Workflow Automation Engine	Workflow DAG runner, triggers, step execution, durable state
RAG / Knowledge Base Service	File ingestion, chunking, embedding, vector search, reranking
Memory Service	Short-term (context window), long-term (vector DB), extraction, retrieval
File Ingestion Pipeline	Parse PDF/DOCX/TXT/CSV, chunk, embed, store in vector DB
Billing & Usage Metering	Stripe subscriptions, token counting, plan enforcement, invoices
Admin / Analytics	Usage dashboards, cost tracking, model metrics, user management
Notification Service	Email/in-app notifications for workflow completion, approvals, errors
Integration Service	OAuth connectors to Slack/Gmail/Notion/Sheets/CRMs/webhooks

3. High-Level System Architecture

Architecture Overview

CAT AI uses a modular monolith for MVP, with clear internal service boundaries that can be extracted into microservices as scale demands. The system is split into synchronous (HTTP/SSE) and asynchronous (queue-based worker) execution paths.

ASCII Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              CLIENT LAYER                                │
│  Next.js Web App   ·   Mobile (future)   ·   Public API consumers        │
└──────────────────────────────────────────────────────────────────────────┘
                              │ HTTPS / WSS / SSE
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                AWS CloudFront + WAF (CDN / DDoS protection)              │
└──────────────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                          FastAPI API Gateway                             │
│  /auth  /chat  /conversations  /files  /knowledge  /workflows            │
│  /integrations  /webhooks  /usage  /billing  /admin                      │
│                                                                          │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────────────┐         │
│  │ Auth Service │   │ Chat Service │   │ Workflow Service     │         │
│  └──────────────┘   └──────────────┘   └──────────────────────┘         │
│                          │                        │                      │
│  ┌──────────────────────────────────────────────┐  │                    │
│  │          AI Orchestration Engine             │  │                    │
│  │   Prompt Assembly → Model Router             │  │                    │
│  │   → Provider Adapter → Stream Back           │  │                    │
│  └──────────────────────────────────────────────┘  │                    │
│                  │                                 │                    │
│  ┌──────────────────────┐   ┌──────────────────────────────────┐        │
│  │ Model Provider Layer │   │ Worker Queue (Redis/SQS)         │        │
│  │ OpenAI · Anthropic   │   │ File Ingestion · Workflow Exec   │        │
│  │ Groq · Gemini · vLLM │   │ Memory Extraction · Webhooks     │        │
│  └──────────────────────┘   └──────────────────────────────────┘        │
└──────────────────────────────────────────────────────────────────────────┘
                              │ reads/writes
                              ▼
┌──────────────┐  ┌──────────────────────┐  ┌──────────────┐  ┌──────────────┐
│ Redis        │  │ PostgreSQL (RDS)     │  │ pgvector     │  │ S3 Files     │
│ Cache        │  │ Primary DB           │  │ Vectors      │  │ Uploads      │
│ Sessions     │  │ All entities         │  │ Embeddings   │  │ Exports      │
│ Queues       │  └──────────────────────┘  └──────────────┘  └──────────────┘
└──────────────┘
```

Synchronous vs Asynchronous Flows

Flow Type	Examples
Synchronous (HTTP/SSE)	Chat responses (streaming), auth, CRUD operations, billing portal
Asynchronous (Queue/Worker)	File ingestion, memory extraction, workflow execution, email notifications

Webhook-triggered async	Inbound webhooks trigger workflow runs via queue
Scheduled async	Cron workflows, usage rollups, summary jobs

Service Boundaries

In MVP, all services live within a single FastAPI process organized as Python packages. The boundaries are enforced by module structure, not network calls. Extraction to separate services happens at scale.

- api/ — FastAPI routers, request validation, auth middleware
- services/ — Business logic (chat, workflow, rag, memory, billing)
- workers/ — Celery/ARQ task handlers for async processing
- ai/ — Model provider adapters, orchestration, prompt assembly
- db/ — SQLAlchemy models, migrations (Alembic), repository pattern

4. Frontend Architecture

Next.js App Structure

```
apps/web/
├── app/                          # Next.js 14 App Router
│   ├── (auth)/                   # login, register, forgot-password
│   ├── (dashboard)/              # authenticated routes
│   │   ├── chat/[conversationId] # main chat interface
│   │   ├── knowledge/            # knowledge base management
│   │   ├── workflows/            # workflow builder + runs
│   │   ├── integrations/         # OAuth connectors
│   │   ├── settings/             # workspace + user settings
│   │   └── admin/                # admin dashboard (role-gated)
│   └── api/                      # Next.js route handlers (thin proxy only)
├── components/
│   ├── chat/                     # MessageList, ChatInput, StreamingMessage
│   ├── workflow/                 # WorkflowCanvas, StepPalette, RunLogs
│   ├── knowledge/                # FileUploader, KBList, SourceCitations
│   └── ui/                       # shadcn/ui base components
├── lib/
│   ├── api.ts                    # Axios/fetch client with auth headers
│   ├── streaming.ts              # SSE parser, token accumulator
│   └── store/                    # Zustand stores
└── hooks/                        # useChat, useWorkflow, useKnowledge
```

Key UI Patterns

UI Area	Design Approach
Chat interface	3-pane layout: sidebar (conversations) + main (messages) + context panel
Streaming UX	SSE with token-by-token rendering; show spinner until first token; error retry
File upload UX	Drag-and-drop with progress bar, chunked upload to S3 presigned URL
Workflow builder	React Flow canvas for DAG visualization; node palette sidebar
Team/workspace UX	Workspace switcher in nav; workspace-scoped data everywhere
Admin dashboard	Usage charts (Recharts), user table, model cost breakdown
Error/retry UX	Toast notifications; inline retry button; offline detection banner

State Management

Zustand for global client state (auth, workspace, conversation list). React Query (TanStack Query) for server state (API data fetching, caching, invalidation). SSE streaming state managed locally in useChat hook. No Redux — too complex for this surface.

5. Backend / API Architecture

FastAPI Service Layout

```
apps/api/
├── main.py                        # FastAPI app factory, middleware, CORS
├── api/
│   ├── v1/
│   │   ├── auth.py                # /auth routes
│   │   ├── chat.py                # /chat streaming endpoint
│   │   ├── conversations.py
│   │   ├── files.py
│   │   ├── knowledge.py
│   │   ├── workflows.py
│   │   ├── integrations.py
│   │   ├── webhooks.py
│   │   ├── usage.py
│   │   ├── billing.py
│   │   └── admin.py
├── services/                      # Business logic layer
├── ai/                            # Orchestration + adapters
├── workers/                       # Celery tasks
├── db/                            # Models, migrations, repos
└── middleware/                    # Auth, rate limit, tenant, logging
```

Streaming Strategy

Chat responses use Server-Sent Events (SSE) via FastAPI's StreamingResponse with EventSourceResponse. Each token chunk is sent as a data: JSON event. The frontend SSE client accumulates tokens. Full message is committed to DB after stream completes.

WebSockets are reserved for future real-time collaboration features. SSE is simpler, HTTP-compatible, and sufficient for one-directional AI response streaming.

Auth & RBAC

- JWT access tokens (15-minute expiry) + refresh tokens (30-day, stored in HttpOnly cookie)
- Roles: owner > admin > member > viewer — enforced at service layer, not just route layer
- Workspace-scoped RBAC: a user can have different roles in different workspaces
- API keys for programmatic access (hashed in DB, never stored in plaintext)

Multi-Tenant Support

All data models include a workspace_id foreign key. Every query is scoped by workspace_id extracted from the JWT claims. No cross-workspace data leakage is possible at the ORM layer. Row-Level Security (RLS) can be added in PostgreSQL as an additional layer for enterprise tier.

Rate Limiting

Rate limiting is enforced per user and per workspace using Redis sliding window counters. Limits are plan-aware: free tier gets 20 requests/minute, pro tier gets 100, team tier gets 500. The AI inference endpoint has separate, stricter limits based on token budget.

6. AI Orchestration Architecture

Prompt Assembly Pipeline

```
PROMPT ASSEMBLY ORDER (token budget consumed in order of priority)

1. System Prompt      [ workspace system prompt + guardrails ]  ~500 tokens
2. Long-term Memory   [ top-k relevant memories from vector DB ]  ~200 tokens
3. RAG Context        [ retrieved document chunks if tool called ] ~1500 tokens
4. Conversation Hist  [ recent messages, summarized if needed ]   ~2000 tokens
5. Tool Results       [ results from any tool calls ]             ~500 tokens
6. User Message       [ current user input ]                       ~500 tokens
                                                        TOTAL: ~5200 tokens

Remaining context window allocated to model response (max_tokens).
If history exceeds budget → summarize older turns → store summary → truncate.
```

Model Routing Logic

Routing Signal	Action
User selected model	Use explicitly selected model regardless of routing
Workspace default model	Use workspace-configured default model
Tool calling required	Route to models that support function calling (GPT-4o, Claude 3.5+)
Cost optimization mode	Route simple queries to Groq Llama3 (fast + cheap)
Long context needed	Route to Gemini 1.5 Pro (1M token context)
Primary provider down	Automatic fallback to secondary provider (circuit breaker)

Context Window Management

- Track running token count with provider-specific tokenizers (tiktoken for OpenAI, estimate for others)
- When conversation exceeds 70% of context limit: summarize oldest N messages into a single summary message using a cheap/fast model (Groq Llama3)
- Summary is stored as a synthetic message in the conversation history
- RAG and memory context are trimmed last (user message is always preserved)

Response Streaming Design

The API streams provider response tokens directly to the client via SSE. The orchestration layer acts as a transparent proxy with added logic: token counting, guardrail screening, tool call detection. Tool calls interrupt streaming, execute the tool, inject results, and resume streaming from the model.

Guardrails & Moderation

Content moderation runs asynchronously via OpenAI Moderation API or a local classifier for user inputs. Workspace-level system prompts define behavioral guardrails. Output is never blocked mid-stream — moderation flags are logged and surfaced to admins post-hoc (non-blocking approach for latency preservation).

7. Workflow Automation Engine

Workflow Model

Workflows are defined as versioned JSON/YAML DAGs. Each workflow has a trigger, a set of steps, and edges between steps. Execution state is persisted to PostgreSQL, making it durable across worker restarts.

```
workflow: {
  id, workspace_id, name, description,
  trigger: { type: 'webhook' | 'cron' | 'manual' | 'event', config: {} },
  steps: [
    { id, type: 'llm' | 'condition' | 'api_call' | 'retrieval' |
            'approval' | 'transform' | 'loop' | 'tool',
      config: {}, inputs: {}, outputs: {} }
  ],
  edges: [{ from: stepId, to: stepId, condition?: string }],
  version: number
}
```

Step Types

Step Type	Description
LLM Step	Send prompt to AI model, receive response, extract structured output
Condition Step	Branch workflow based on JSONLogic/expression evaluation
API Call Step	HTTP request to external service with auth (OAuth, API key, Basic)
Retrieval Step	Query knowledge base, inject retrieved chunks into context
Approval Step	Pause execution, notify human, wait for approve/reject/modify response
Transform Step	Apply JSONata/JMESPath transformation to reshape data between steps
Loop Step	Iterate over an array of items, run sub-workflow for each
Tool Step	Execute a registered tool (web search, email, Slack, etc.)

Execution Engine Design

- Each workflow run creates a workflow_run record with status: pending → running → completed/failed
- Each step creates a workflow_step_run record with input, output, status, and timing
- Execution engine is a Celery worker task; tasks are chained/grouped per step dependencies
- Durable state: if worker dies mid-run, the run can be resumed from the last completed step
- Retry strategy: each step has configurable max_retries and backoff (default: 3 retries, exponential backoff)
- Timeout: each step has a configurable timeout (default: 30s for LLM, 10s for API calls)
- Dead-letter queue: steps that exhaust retries are moved to DLQ for manual inspection
- Idempotency: step execution IDs are deduplicated; re-queued steps check for existing output

Human-in-the-Loop Approvals

Approval steps pause workflow execution and emit a notification (email + in-app) to designated approvers. The workflow_step_run status is set to 'awaiting_approval'. A signed approval URL in the notification allows approve/reject without requiring login. Approval timeout (default 24h) can auto-approve or auto-reject.

8. Memory Architecture

Memory Type	Implementation
Short-term (in-context)	Last N messages in conversation history injected into prompt context
Session memory	Conversation summary stored in DB, injected as context after summarization
Long-term (semantic)	Extracted facts stored as vector embeddings in pgvector memories table
Workspace shared memory	Team-level facts/preferences stored in workspace-scoped memory namespace

Memory Extraction Pipeline

After each AI response, a lightweight background task runs a memory extraction prompt against the last exchange. The extractor identifies facts worth remembering (user preferences, stated constraints, decisions made, important context). Extracted facts are embedded and stored in the memories table with workspace_id and user_id scoping.

Memory Retrieval

At prompt assembly time, the top-k most semantically relevant memories (cosine similarity) are retrieved and injected into the system prompt section. Recency and relevance are balanced using a weighted score: score = 0.7 * similarity + 0.3 * recency_decay.

Privacy Controls

- Users can view all stored memories via the Settings > Memory page
- Users can delete individual memories or clear all memories
- Memory extraction can be disabled per-user or per-workspace
- Memory data is workspace-isolated; no cross-workspace leakage
- GDPR deletion request triggers cascade deletion of all memory records

9. RAG / Knowledge Base Architecture

File Ingestion Pipeline

```
INGESTION PIPELINE

1. Upload    → User uploads file → Stored in S3 (raw)
2. Parse     → Worker picks up job → Extracts text
             → PDF: pdfminer/pymupdf  DOCX: python-docx
             → TXT/CSV/XLSX: direct   Web URL: crawl4ai
3. Clean     → Remove headers/footers, normalize whitespace, fix encoding
4. Chunk     → Split into overlapping chunks (512 tokens, 50-token overlap)
             → Semantic chunking for coherence (optional, higher quality)
5. Embed     → Call embedding model (text-embedding-3-small by default)
6. Store     → Insert chunks + vectors into document_chunks + pgvector
7. Index     → Update knowledge_base metadata, trigger re-index if update
8. Notify    → Mark file as 'ready', notify user
```

Chunking Strategy

Strategy	Use Case
Fixed-size chunking (512 tokens, 50 overlap)	Default; fast, predictable, good for most docs
Sentence-boundary chunking	Better for narrative text, preserves semantic units
Semantic chunking	Groups semantically similar sentences; highest quality; slowest
Hierarchical chunking	Creates parent chunks + child chunks; enables both broad and precise retrieval

Retrieval Strategy

Hybrid retrieval combines vector similarity search (pgvector cosine distance) with BM25 keyword search (PostgreSQL full-text search). Results from both are merged and reranked using a cross-encoder reranker (Cohere Rerank or open-source reranker). Final top-k chunks are injected into prompt with source metadata for citations.

pgvector vs Qdrant Tradeoff

Factor	Decision
MVP simplicity	pgvector: one less service to manage, same Postgres instance
Query performance (<1M chunks)	pgvector HNSW index is sufficient for MVP scale
Query performance (>1M chunks)	Migrate to Qdrant for better ANN performance and filtering
Metadata filtering	pgvector uses SQL WHERE clauses; Qdrant has native payload filtering

Migration path	Dual-write to both during migration; switch read traffic; deprecate pgvector

10. Database Architecture

PostgreSQL Data Model

Table	Purpose	Key Columns	Indexes
`users`	Core user identity	id, email, hashed_password, name, avatar_url, is_active, created_at, last_login	email (unique)
`organizations`	Workspace/team entity	id, name, slug, plan, settings (jsonb), created_at, owner_id → users	slug (unique)
`memberships`	User ↔ org relationship	id, user_id → users, org_id → organizations, role (enum), joined_at	user_id + org_id (unique)
`sessions`	JWT refresh token store	id, user_id → users, token_hash, device_info, expires_at, revoked_at	token_hash
`conversations`	Chat session container	id, org_id, user_id, title, model_id, system_prompt, summary, metadata (jsonb), created_at	org_id + created_at
`messages`	Individual chat messages	id, conversation_id, role (user/assistant/tool), content, tokens_in, tokens_out, model_id, latency_ms, created_at	conversation_id + created_at
`tool_calls`	Tool call trace within message	id, message_id, tool_name, input (jsonb), output (jsonb), status, duration_ms	message_id
`memories`	Long-term memory facts	id, org_id, user_id, content, embedding (vector), importance_score, source_conversation_id, created_at	embedding (HNSW index), org_id + user_id
`files`	Uploaded file metadata	id, org_id, user_id, filename, mime_type, s3_key, size_bytes, status (enum), error_message, created_at	org_id + status
`knowledge_bases`	Vector KB container	id, org_id, name, description, embedding_model, chunk_strategy, doc_count, created_at	org_id
`documents`	Parsed document in KB	id, kb_id, file_id, title, source_url, char_count, chunk_count, status, created_at	kb_id
`document_chunks`	Embedding unit	id, document_id, kb_id, content, embedding (vector), chunk_index, token_count, metadata (jsonb)	embedding (HNSW), kb_id, document_id
`workflows`	Workflow definition	id, org_id, name, description, trigger (jsonb), is_active, created_by → users, created_at	org_id
`workflow_versions`	Versioned workflow snapshot	id, workflow_id, version, definition (jsonb), published_at	workflow_id + version

workflow_runs	Execution instance	id, workflow_id, version, trigger_type, status, started_at, completed_at, error, context (jsonb)	workflow_id + status, started_at
workflow_step_runs	Per-step execution record	id, run_id, step_id, status, input (jsonb), output (jsonb), started_at, completed_at, retry_count	run_id, step_id
integrations	OAuth/API connections	id, org_id, user_id, provider (slack/gmail/etc), encrypted_credentials, scopes, expires_at, created_at	org_id + provider
webhook_events	Inbound webhook log	id, org_id, source, headers (jsonb), payload (jsonb), workflow_run_id, processed_at, created_at	org_id + created_at
usage_logs	Per-request metering	id, org_id, user_id, resource_type, model_id, tokens_in, tokens_out, cost_usd, duration_ms, created_at	org_id + created_at (partitioned)
subscriptions	Billing subscription	id, org_id, stripe_subscription_id, stripe_customer_id, plan, status, current_period_end, created_at	org_id (unique), stripe_subscription_id
invoices	Billing invoice record	id, org_id, stripe_invoice_id, amount_usd, status, period_start, period_end, pdf_url, paid_at	org_id + period_start
audit_logs	Security audit trail	id, org_id, user_id, action, resource_type, resource_id, ip_address, metadata (jsonb), created_at	org_id + created_at (partitioned)
notifications	In-app notifications	id, user_id, type, title, body, read_at, action_url, created_at	user_id + read_at

When to Use Redis / S3

Storage Layer	Use Cases
Redis	JWT refresh token blacklist, rate limit counters, session cache, SSE connection state, Celery task queue, real-time usage counters
S3 (Object Storage)	Raw uploaded files, exported conversation archives, workflow definition backups, invoice PDFs, model response audit dumps
pgvector (PostgreSQL)	All embedding vectors (memories, document_chunks) — colocated with metadata for easy filtering

11. Model Provider Abstraction Layer

Adapter Interface

All providers implement a unified BaseProvider abstract class. This ensures the orchestration layer is completely provider-agnostic. Swapping models requires only changing the provider field, not the orchestration code.

```
class BaseProvider:
  async def chat_completion(request: ChatRequest) -> ChatResponse
  async def stream_completion(request: ChatRequest) -> AsyncIterator[Token]
  async def embed(texts: List[str], model: str) -> List[List[float]]
  async def tool_call(request: ChatRequest, tools: List[Tool]) ->
ToolCallResponse
  def health_check() -> ProviderHealth
  def token_count(messages: List[Message], model: str) -> int

Implementations: OpenAIProvider, AnthropicProvider, GroqProvider,
                 GeminiProvider, VLLMProvider
```

Provider Normalization

Capability	Normalization Strategy
Chat format	Normalize to OpenAI message format internally; translate to each provider's schema
Streaming	Normalize all SSE/streaming protocols to AsyncIterator[TokenChunk]
Tool calling	Normalize OpenAI function_call, Anthropic tool_use to unified ToolCall schema
Embeddings	Normalize to List[float]; handle batching differences per provider
Error codes	Map provider HTTP errors to unified ProviderError with retry hints
Token counting	tiktoken for OpenAI/Groq; estimated 1:3.5 char ratio for Anthropic/Gemini

Circuit Breaker & Failover

Each provider has a circuit breaker (half-open/open/closed states) implemented with Redis counters. If a provider returns >5 errors in 60 seconds, the circuit opens and traffic is rerouted to the fallback provider. Circuit retries after 30 seconds. Failover order: OpenAI → Anthropic → Groq → error.

Cost Tracking

After each completion, input and output token counts are recorded in usage_logs with the per-token cost for that model. Costs are aggregated daily per workspace. Alerts fire when workspace spending exceeds configurable thresholds. Cost data feeds into the billing metering pipeline.

12. Tool / Agent Framework

Tool Registry Design

Tools are registered in a central ToolRegistry with a JSON Schema definition compatible with OpenAI function calling and Anthropic tool use. Each tool has an execution handler, a permission scope, and optional sandboxing requirements.

```
ToolDefinition {
  name: str,          # unique tool identifier
  description: str,   # shown to the model
  parameters: JsonSchema,
  permission: ToolPermission,  # workspace, user, admin
  sandbox: bool,      # run in isolated sandbox
  timeout_ms: int,
  retry_policy: RetryPolicy
}
```

Built-in Tools

Tool	Description
web_search	Serper/Tavily API for real-time web search with snippet extraction
calculator	Sandboxed math expression evaluator (no LLM hallucination on math)
knowledge_retrieval	Query workspace knowledge bases; injects citations
memory_read / write	Read/write user long-term memory explicitly
send_email	Via Gmail OAuth integration or SMTP
slack_message	Post to Slack channel via OAuth bot token
notion_create	Create Notion page via Notion API integration
google_sheets	Read/write Google Sheets via Sheets API
webhook_call	Outbound HTTP call to configured webhook URL
code_executor	Run Python/JS in isolated Docker sandbox; return stdout/stderr
browser_automation	Playwright-based headless browser for web scraping/interaction

Tool Execution Lifecycle

6. Model returns tool_call in response (detected in stream parser)
7. Streaming paused; tool call extracted and validated against schema
8. Permission check: user/workspace authorized for this tool
9. Tool executed in sandbox (if required) with timeout
10. Result stored in tool_calls table with input/output/duration
11. Result injected into message history as tool_result role
12. Model call resumed with tool result in context
13. Final response streamed back to user

Prompt Injection Defense

- Tool descriptions and schemas are immutable at runtime — not user-supplied
- Tool inputs are validated against JSON Schema before execution
- Code execution tools run in Docker containers with no network access, read-only filesystem, and resource limits (CPU/memory/time)
- Webhook URLs are validated against a workspace-level allowlist
- All tool outputs are treated as untrusted content — not injected as system instructions

13. Security Architecture

Security Layer	Implementation
Authentication	JWT (access 15m) + refresh token (30d, HttpOnly cookie); bcrypt password hashing
OAuth / SSO	Google + GitHub OAuth2 (MVP); SAML 2.0 / OIDC SSO for enterprise tier
Secrets management	AWS Secrets Manager for API keys; never in env files or DB plaintext
User credential storage	OAuth integration tokens encrypted with AES-256-GCM; key in AWS KMS
Encryption at rest	AWS RDS encryption + S3 server-side encryption (SSE-S3)
Encryption in transit	TLS 1.3 everywhere; HTTPS enforced via CloudFront
Workspace isolation	All queries scoped by workspace_id; ORM-level enforcement
File upload security	ClamAV malware scan on upload; MIME type validation; S3 bucket policy blocks public access
RBAC	Role enum: owner/admin/member/viewer; checked in service layer before any mutation
Audit logging	All auth events, data mutations, billing events written to audit_logs
Webhook verification	HMAC-SHA256 signature verification on all inbound webhooks
GDPR / data deletion	User deletion triggers cascading soft-delete + async hard-delete job for all PII

14. DevOps / Deployment Architecture

AWS Infrastructure

```
AWS DEPLOYMENT TOPOLOGY

Route 53 (DNS)
  └── CloudFront (CDN + WAF) ──→ S3 (Next.js static assets)
                              └→ ALB (Application Load Balancer)
                                   ├── ECS Fargate (API — FastAPI)
                                   └── ECS Fargate (Worker — Celery)

RDS PostgreSQL (Multi-AZ, encryption at rest)
  └── pgvector extension enabled

ElastiCache Redis (cluster mode, 2 shards)
  ├── Celery task queue
  └── Cache / rate limiting

S3 Bucket: cat-ai-files (private, versioned)
S3 Bucket: cat-ai-assets (public, CDN-fronted)

Secrets Manager: all API keys, DB creds, JWT secrets
ECR: Docker image registry
CloudWatch: Logs + Metrics + Alarms
```

CI/CD Pipeline

- GitHub Actions: test → build → push Docker image to ECR → deploy to ECS (blue-green)
- 3 environments: dev (single-container), staging (production topology, smaller instances), prod
- Database migrations (Alembic) run as a separate ECS task before deployment, with rollback on failure
- Frontend deployed to S3 + CloudFront via CDK or Terraform; cache invalidation on deploy
- Canary deployments for API: shift 10% traffic to new task, monitor error rate, then complete rollout

Backup & Disaster Recovery

- RDS: automated daily snapshots (30-day retention) + point-in-time recovery (PITR) enabled
- S3: versioning enabled; cross-region replication for enterprise tier
- Redis: ElastiCache automatic backup + AOF persistence for queue durability
- RTO target: 1 hour (multi-AZ RDS failover is automatic); RPO target: 1 minute (PITR)

15. Observability & Analytics

Observability Layer	Tool / Approach
Structured logging	Python structlog → CloudWatch Logs; JSON format; correlation IDs on every request
Metrics	Prometheus (exposed from FastAPI) → Grafana dashboards; model latency p50/p95/p99
Distributed tracing	OpenTelemetry SDK → AWS X-Ray or Jaeger; trace per chat request + tool call
LLM observability	Langfuse (open-source) for prompt traces, token usage, model comparison
Error tracking	Sentry for both frontend (Next.js) and backend (FastAPI) error capture
Product analytics	PostHog (open-source, self-hostable) for feature usage, funnel analytics
Cost tracking	Custom dashboard from usage_logs table: $/1k tokens per model per workspace
Workflow metrics	Run success rate, step failure breakdown, avg duration per workflow
Uptime monitoring	Better Uptime / AWS Route 53 health checks on /health endpoint

16. Billing & Monetization Architecture

Plan Structure

Plan	Limits	Price
Free	50 messages/day, 1 workspace, 10MB storage, OpenAI/Groq only	$0
Pro	Unlimited messages, 1 workspace, 5GB storage, all models, basic automations	$20/mo
Team	Unlimited, 5 workspaces, 50GB storage, all models, full automations, RBAC	$79/mo
Enterprise	Custom limits, SSO, SAML, dedicated support, SLAs, custom retention	Custom

Stripe Integration

- Stripe Checkout for plan upgrades; Stripe Customer Portal for plan management
- Stripe webhooks (invoice.paid, customer.subscription.updated, etc.) received at /webhooks/stripe
- Metered billing via Stripe Usage Records: token usage reported hourly for pay-per-use add-ons
- Feature gating: plan checked on every request via Redis cache (subscription status cached for 5 minutes)
- Dunning: Stripe handles failed payment retries and cancellation notifications

17. Scalability Strategy

Scale Stage	Users / Load	Infrastructure Changes
MVP	0–1K users	1 ECS task (API), 1 ECS task (Worker), db.t3.medium RDS, cache.t3.micro Redis
Growth	1K–10K users	ECS auto-scaling (2–5 tasks), db.r6g.large RDS, Multi-AZ RDS, Redis cluster
Scale	10K–100K users	ECS (5–20 tasks), RDS read replicas, Qdrant for vectors, SQS for queue, CDN optimization
Hyper-scale	100K–1M+ users	EKS (Kubernetes), DB sharding/partitioning, separate vector cluster, global CDN, multi-region

Key Scaling Decisions

- usage_logs and audit_logs tables partitioned by month from day 1 to avoid single-table bloat
- document_chunks table partitioned by kb_id for efficient per-knowledge-base queries
- Redis for all hot-path reads (subscription status, rate limits, session cache) — never hit DB for these
- Streaming responses don't buffer in server — always proxied directly to client to minimize memory pressure
- Provider rate limits handled with token bucket per provider in Redis; requests are queued not dropped

18. Recommended Tech Stack

MVP Stack (Speed to Ship)

Layer	Technology
Frontend	Next.js 14, TypeScript, Tailwind CSS, shadcn/ui, Zustand, TanStack Query
Backend API	FastAPI, Python 3.12, Pydantic v2, SQLAlchemy 2.0, Alembic
Database	PostgreSQL 16 (RDS), pgvector extension
Cache / Queue	Redis 7 (ElastiCache), Celery with Redis broker
Object Storage	AWS S3
LLM Providers	OpenAI SDK + Anthropic SDK (MVP); add Groq + Gemini in Phase 2
Auth	python-jose for JWT, bcrypt, Google OAuth2
Billing	Stripe Python SDK
Deployment	Docker, ECS Fargate, GitHub Actions CI/CD
Monitoring	Sentry, CloudWatch, Langfuse (self-hosted or cloud)

Production Stack (Full Platform)

Layer	Technology
Vector DB	Qdrant (self-hosted on EKS or Qdrant Cloud) replacing/alongside pgvector
Queue	AWS SQS + Celery for better scalability and dead-letter support
Observability	OpenTelemetry + Grafana + Loki stack (or Datadog for managed)
Code Execution	Docker-in-Docker sandbox or AWS Lambda for isolated code execution
Self-hosted Models	vLLM on EKS GPU nodes (g4dn.xlarge) for cost-optimized inference
Analytics	PostHog (self-hosted), ClickHouse for usage analytics at scale
SSO	Auth0 or Keycloak for enterprise SAML/OIDC

19. Monorepo / Project Structure

```
cat-ai/
├── apps/
│   ├── web/                    # Next.js frontend
│   ├── api/                    # FastAPI backend
│   ├── worker/                 # Celery worker (can share api code)
│   └── admin/                  # Internal admin tooling (optional)
├── packages/
│   ├── ai-sdk/                 # Provider adapters + orchestration engine
│   ├── workflow-engine/        # DAG runner, step executors, durable state
│   ├── tool-registry/          # Tool definitions + execution handlers
│   ├── rag-pipeline/           # File parsing, chunking, embedding pipeline
│   ├── shared-types/           # Pydantic models + TypeScript types (codegen)
│   ├── ui/                     # Shared React component library
│   └── prompts/                # Versioned prompt templates
├── infra/
│   ├── terraform/              # AWS infrastructure as code
│   └── docker/                 # Dockerfiles, compose files
├── scripts/
│   ├── seed.py                 # DB seed for dev
│   └── migrate.sh              # Alembic migration runner
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/                    # Playwright E2E tests
└── docs/
    ├── architecture/           # This document
    └── api/                    # OpenAPI spec + guides
```

20. Sample API Surface

Method	Endpoint	Description
POST	`/auth/register`	Register new user
POST	`/auth/login`	Login, receive JWT + refresh token
POST	`/auth/refresh`	Refresh access token
GET	`/auth/me`	Get current user profile
POST	`/chat/stream`	SSE streaming chat (main AI endpoint)
GET	`/conversations`	List conversations (paginated)
POST	`/conversations`	Create new conversation
GET	`/conversations/{id}/messages`	Get message history
DELETE	`/conversations/{id}`	Delete conversation
POST	`/files/upload`	Get presigned S3 URL, trigger ingestion
GET	`/files`	List files in workspace
GET	`/knowledge`	List knowledge bases
POST	`/knowledge`	Create knowledge base
POST	`/knowledge/{id}/query`	Query knowledge base directly
GET	`/workflows`	List workflows
POST	`/workflows`	Create workflow
POST	`/workflows/{id}/run`	Trigger manual workflow run
GET	`/workflow-runs/{id}`	Get run status + step details
POST	`/integrations/connect`	Initiate OAuth flow for integration
GET	`/webhooks`	List registered webhooks
POST	`/webhooks/{provider}`	Inbound webhook receiver
GET	`/usage/summary`	Usage stats for current billing period
POST	`/billing/portal`	Create Stripe Customer Portal session
GET	`/admin/users`	List all users (admin only)
GET	`/admin/metrics`	Platform-wide metrics (admin only)

21. End-to-End Request Flows

A. Normal Chat Request

```
1. User types message → Next.js opens SSE connection to POST /chat/stream
2. API middleware: validate JWT → extract workspace_id, user_id
3. Rate limit check: Redis sliding window counter
4. Load conversation history from DB (last N messages)
5. Load top-k memories from pgvector (semantic search on user message)
6. Load workspace system prompt from DB
7. Orchestration: assemble prompt with token budget management
8. Select model: workspace default or user-selected
9. Call provider adapter → begin streaming response
10. Stream tokens via SSE to client (data: {token} events)
11. On stream complete: save assistant message to DB
12. Async: enqueue memory extraction task → worker processes later
13. Async: write usage_log record (tokens, cost, latency)
```

B. Chat Request with Tool Call

```
1-9. Same as normal chat request
10. Model returns tool_call in stream (detected by stream parser)
11. Streaming paused; tool_call extracted
12. Tool permission check: is this tool allowed for this user/workspace?
13. Tool executed (e.g., web_search API call) with timeout
14. Tool result stored in tool_calls table
15. Tool result injected as tool_result message in conversation
16. Second model call with updated history + tool result
17. Final response streamed to client
18. All messages (user + tool_call + tool_result + assistant) saved to DB
```

C. Chat Request with RAG Retrieval

```
1-6. Same as normal chat request
7. RAG retrieval: embed user message → cosine search in pgvector
8. Hybrid retrieval: combine vector results with BM25 keyword results
9. Rerank: cross-encoder reranker scores and filters top chunks
10. Top-k chunks injected into prompt context with source metadata
11. Model responds with citations referencing chunk source_ids
12. Response includes [source: filename, page X] annotations
13. Frontend renders citations as inline badges linking to source files
```

D. Workflow Triggered by Webhook

```
1. External system sends HTTP POST to /webhooks/{provider}?token=xxx
2. HMAC-SHA256 signature verified against stored webhook secret
3. Webhook event stored in webhook_events table (idempotency check)
4. Matching active workflows found by provider + event type filter
5. For each matching workflow: create workflow_run record (status=pending)
6. Celery task enqueued: execute_workflow(run_id, trigger_payload)
7. Worker picks up task: load workflow version definition from DB
8. Execute steps in topological order:
   - LLM step: call AI orchestration engine
   - API call step: HTTP request to external service
   - Condition step: evaluate expression, choose branch
9. Each step result stored in workflow_step_runs table
```

```
10. On completion: update workflow_run status, fire notification
```

E. Scheduled Workflow Run

```
1. Celery Beat scheduler checks cron_workflows every minute
2. Workflows whose next_run_at <= now() are selected
3. workflow_run created (trigger_type=cron), task enqueued
4. Same execution path as webhook-triggered run (steps D.7-D.10)
5. After run: next_run_at updated = now() + cron_interval
```

F. Human Approval Workflow Step

```
1. Execution engine reaches an approval step
2. workflow_step_run status set to 'awaiting_approval'
3. Notification sent to designated approvers (email + in-app)
4. Email contains: summary of workflow context + signed approval URL
5. Signed URL = JWT with { run_id, step_id, exp: now + 24h }
6. Approver clicks approve/reject → POST /workflow-runs/{id}/approve
7. API validates signed URL, updates step status
8. If approved: workflow resumes from next step
9. If rejected: workflow run marked failed; rejection reason logged
10. If timeout (24h): auto-action based on step config (approve/reject/wait)
```

G. Memory Extraction After Response

```
1. Chat response streaming completes; message saved to DB
2. Async Celery task enqueued: extract_memories(message_id)
3. Worker loads the exchange (user message + assistant response)
4. Memory extraction prompt sent to cheap/fast model (Groq Llama3):
   'Extract any facts, preferences, or decisions from this conversation'
5. Extracted facts returned as JSON array
6. Each fact embedded using text-embedding-3-small
7. Deduplication: cosine similarity check against existing memories
   (skip if similarity > 0.95 — already known)
8. New memories inserted into memories table (user_id, org_id scoped)
9. Total memory count checked against plan limit
```

22. Risks & Tradeoffs

Decision	Option A	Option B / Recommendation
Architecture style	Microservices from day 1 (complex, over-engineered for MVP)	Modular monolith first → extract services at scale ✓
Backend framework	NestJS (TypeScript, more ceremony, strong DI)	FastAPI (Python, faster iteration, AI ecosystem fit) ✓
Vector DB	Qdrant (purpose-built, better at scale, extra infra)	pgvector for MVP → migrate to Qdrant at 500K+ chunks ✓
LLM hosting	Self-hosted vLLM (control, cost at scale, GPU infra complexity)	Hosted APIs for MVP → add vLLM path for cost optimization later ✓
Orchestration	LangChain/LlamaIndex (opinionated, leaky abstractions)	Custom lightweight orchestration layer with provider adapters ✓
Workflow engine	Temporal.io (enterprise-grade, complex setup)	Custom Celery-based engine for MVP → evaluate Temporal at scale ✓
Auth	Auth0 (managed, easy SSO, cost at scale)	Custom JWT for MVP → integrate Auth0 for enterprise SSO ✓
Cost vs quality	Always use GPT-4o (high quality, expensive)	Route by task complexity: Groq for simple, GPT-4o for complex ✓

© 2026 CAT AI · Confidential

23. MVP Phased Blueprint

Phase 1: Core Chat + Auth + Streaming (Weeks 1–3)

Item	Detail
Features	User registration, login, JWT auth · Single workspace per user · Multi-turn chat with streaming SSE · OpenAI + Anthropic model support · Chat history stored in DB
Services touched	FastAPI API (auth + chat routes), Next.js chat UI, PostgreSQL (users, conversations, messages)
DB tables needed	users, sessions, conversations, messages
Infra dependencies	Single ECS task, RDS Postgres, S3 for future use
Main risks	Streaming SSE reliability across proxies/load balancers

Phase 2: History + Model Routing + Workspaces (Weeks 4–5)

Item	Detail
Features	Organizations + memberships + RBAC · Workspace system prompts · Model selection per conversation · Context window management + summarization · Groq + Gemini provider support
Services touched	Org management service, Model routing logic, Summarization background job
DB tables needed	organizations, memberships, + updated conversations
Infra dependencies	Celery worker added, Redis for queue and cache
Main risks	Token counting accuracy across providers

Phase 3: File Upload + RAG (Weeks 6–8)

Item	Detail
Features	File upload UI + S3 presigned URLs · PDF/DOCX/TXT parsing · Chunking + embedding pipeline · pgvector semantic search · Source citations in responses
Services touched	File ingestion worker, RAG retrieval service, Knowledge base CRUD API
DB tables needed	files, knowledge_bases, documents, document_chunks
Infra dependencies	pgvector enabled on RDS, S3 file bucket
Main risks	Embedding costs at scale; chunking quality

Phase 4: Tool Calling + Integrations (Weeks 9–11)

Item	Detail

Features	Tool registry + execution engine · Web search, calculator, KB retrieval tools · Slack + Gmail OAuth integrations · Tool trace logging · Code execution sandbox
Services touched	Tool execution service, Integration OAuth flow, Sandbox executor
DB tables needed	tool_calls, integrations
Infra dependencies	Docker-in-Docker for sandbox, or AWS Lambda
Main risks	Sandbox security; tool timeout reliability

Phase 5: Workflow Automation Engine (Weeks 12–15)

Item	Detail
Features	Workflow builder UI (React Flow) · Webhook + cron + manual triggers · LLM, condition, API call, approval steps · Human approval notifications · Workflow run logs + observability
Services touched	Workflow execution engine, Approval notification service, Workflow definition API
DB tables needed	workflows, workflow_versions, workflow_runs, workflow_step_runs, webhook_events
Infra dependencies	SQS for reliable workflow queuing, Celery Beat for cron
Main risks	Durable execution reliability; complex DAG edge cases

Phase 6: Billing + Analytics + Hardening (Weeks 16–18)

Item	Detail
Features	Stripe subscriptions + Customer Portal · Plan-based feature gating · Usage dashboard · Langfuse LLM observability · Sentry error tracking · Audit logs · GDPR data deletion
Services touched	Billing service, Usage metering, Admin dashboard
DB tables needed	subscriptions, invoices, usage_logs, audit_logs
Infra dependencies	CloudWatch alarms, WAF rules, RDS Multi-AZ enabled
Main risks	Stripe webhook reliability; usage metering accuracy

24. Final Recommendation

Architecture Summary

CAT AI should be built as a modular monolith using FastAPI (Python) on the backend and Next.js on the frontend, deployed on AWS ECS Fargate. The architecture prioritizes speed to market, clear module boundaries that support future microservices extraction, and a strong AI capability foundation from day one.

Best MVP Architecture

Layer	Choice + Rationale
Frontend	Next.js 14 App Router — SSR for SEO, RSC for performance, one deployment target
Backend	Single FastAPI app with internal service modules — ship fast, extract later
Database	Single RDS PostgreSQL instance with pgvector — one DB to manage
Queue	Redis + Celery — battle-tested, easy local dev, no extra AWS services
Storage	S3 from day 1 — no reason to defer, trivial to set up
LLM	OpenAI + Anthropic adapters — covers 95% of use cases
Deployment	ECS Fargate — serverless containers, no Kubernetes overhead

Best Production Architecture

Layer	Change from MVP + Rationale
Vector DB	Add Qdrant Cloud when chunks > 500K — better ANN, filtering, and scaling
Queue	Migrate to SQS for workflow jobs — better durability, visibility timeout, DLQ
Observability	Full OpenTelemetry → Grafana stack + Langfuse — essential for AI cost management
Auth	Add Auth0 for enterprise SAML/OIDC SSO — accelerates enterprise sales
Models	Add Groq (speed) + Gemini (long context) + vLLM path (cost at scale)
Infrastructure	RDS Multi-AZ + read replicas, ElastiCache cluster, ECS auto-scaling groups

Exact Frameworks & Libraries

```
# BACKEND
fastapi==0.115+          # API framework
```

```
pydantic==2.9+            # Data validation
sqlalchemy==2.0+          # ORM
alembic                   # DB migrations
celery[redis]==5.4+       # Task queue
openai==1.50+             # OpenAI SDK
anthropic==0.35+          # Anthropic SDK
groq                      # Groq SDK
google-generativeai       # Gemini SDK
pgvector                  # pgvector Python client
sentence-transformers     # Local embedding fallback
pdfminer.six + pymupdf    # PDF parsing
python-docx               # DOCX parsing
python-jose[cryptography]# JWT
bcrypt                    # Password hashing
stripe                    # Billing
structlog                 # Structured logging
sentry-sdk                # Error tracking

# FRONTEND
next@14+                  # Framework
typescript                # Type safety
tailwindcss               # Styling
@shadcn/ui                # Component library
zustand                   # Client state
@tanstack/react-query     # Server state
reactflow                 # Workflow canvas
recharts                  # Charts/analytics
eventsource-parser        # SSE client parsing

# INFRA
terraform                 # IaC for AWS
docker + docker-compose   # Local dev
github actions            # CI/CD
```

First 30-Day Architecture-to-Build Roadmap

Days	Milestone
Days 1–3	Finalize this blueprint; set up monorepo; configure local dev (Docker Compose with Postgres + Redis)
Days 4–7	Auth system: register/login/JWT/refresh; user + workspace DB models; Alembic migrations
Days 8–12	Chat foundation: conversation + message models; provider abstraction layer (OpenAI + Anthropic); SSE streaming endpoint
Days 13–16	Frontend: Next.js app shell; auth pages; main chat UI with streaming; conversation sidebar
Days 17–20	Prompt assembly: context window management; workspace system prompts; conversation summarization
Days 21–24	File upload: S3 presigned URLs; file ingestion worker; PDF/DOCX parsing; basic chunking + pgvector
Days 25–27	RAG retrieval: semantic search; hybrid retrieval; citations in responses

Days 28–30	Deploy to AWS ECS (staging); set up CI/CD; Sentry error tracking; basic usage logging; internal demo

CAT AI Architecture Blueprint v1.0 — Ready for Implementation